

FORMAL LANGUAGES AND COMPILERS

prof.s Giovanni Agosta, Luca Breveglieri and Daniele Cattaneo

Exam of MON 19 JANUARY 2026 – Theory Part

WITH SOLUTIONS – FOR TEACHING PURPOSES THE SOLUTIONS ARE WIDELY COMMENTED

LAST NAME (SURNAME) + FIRST NAME:

(capital letters please)

MATRICOLA:

(or PERSON CODE)

SIGNATURE:

TEACHER: ☐ Prof. G. AGOSTA – ☐ Prof. L. BREVEGLIERI – ☐ Prof. D. CATTANEO

INSTRUCTIONS – READ CAREFULLY:

- The exam is open book: textbooks and personal notes are permitted.
- Pencil writing is permitted, except on the cover page (this sheet).
- Please write in the free space left and if necessary continue on the back side of the sheet; do not attach new sheets and do not replace the existing ones.
- Time: lab part 1 h – theory part 2 h.

SCORING AND GRADE ATTRIBUTION:

- The exam is in written form and consists of two parts:

Theory syntax and semantics of languages, divided in four sections: (75% of the final grade)

- 1) regular expressions and finite automata (1 exercise × 10 points)
- 2) syntax analysis and parsing methodologies (1 exercise × 10 points)
- 3) grammar design – translation – semantic analysis (2 exercises × 5 points)
- 4) *bonus* question (1 exercise × 5 points)

Lab compiler design by Flex and Bison (25% of the final grade)

- 1) basic questions (30 points)
- 2) *bonus* question (5 points)

- For the theory part to be valid, the candidate must achieve a grade of at least 5/10 in each of the three sections 1, 2 and 3. If this is not the case, section 4 is not evaluated. Section 4 has no threshold on its own. Correctly answering all the questions in sections 1, 2 and 3 allows the candidate to achieve a grade of 30/30 (but not *cum laude*) for the theory part.
- For the lab part to be valid, the candidate must achieve a grade of at least 15/30 in the basic questions. The bonus question is evaluated only if the threshold is reached.
- To pass the exam, the candidate must succeed in both parts (theory and lab), in one call or more calls separately, but within one year (12 months) between the two parts.
- The final grade is the weighted average of the theory part (75 %) and of the lab part (25 %), and must be $\geq 18/30$ (*cum laude* $\geq 32/30$).

1 Regular expressions and finite automata (10 points)

Consider the regular expression R below, over the three-letter alphabet $\Sigma = \{a, b, c\}$:

$$R = (ab \mid b^+ \mid ca^+)^*$$

Answer the following questions (use the figures / tables / spaces on the next pages):

- (a) List all the strings of length ≤ 2 (less than or equal to 2) generated by the regular expression R (please order the string lexicographically assuming $a \prec b \prec c$).
 - (b) Prove that the regular expression R is ambiguous, by exhibiting at least an ambiguous valid string and its (two or more) derivations. What is the ambiguity degree of R (finite, unlimited or infinite)? Justify your answers.
 - (c) Apply the Berry-Sethi (BS) method to the regular expression R and build a deterministic automaton A equivalent to R . Test the correctness of A with the valid strings found at point (a).
 - (d) Check whether the deterministic automaton A is minimal or not, and if necessary minimize it.
 - (e) Apply the node-elimination method (BMC) to the (minimal) deterministic automaton A and find a regular expression R' equivalent to the regular expression R . Is expression R' still ambiguous?
-

Solution

- (a) Here are all the strings of length ≤ 2 that are generated by the regular expression R , for a total of five strings ordered lexicographically according to $a \prec b \prec c$:

$\varepsilon \quad b \quad ab \quad bb \quad ca$

- (b) The valid string bb is ambiguous. Here are two different derivations of bb (the choices are in red):

$$R \Rightarrow (b^+)^{*=2} \Rightarrow b^{+=1} b^{+=1} \Rightarrow bb$$

$$R \Rightarrow (b^+)^{*=1} \Rightarrow b^{+=2} \Rightarrow bb$$

Such a form of ambiguity is caused by the double iterator component $(b^+)^*$ of expression R . These are the only two derivations of string bb , which thus has a degree of ambiguity equal to 2.

However expression R is (at least) unlimitedly ambiguous, as there is no upper bound to the degree of ambiguity of every string in $L(R)$. In fact, consider the infinitely many valid strings $b^n \in L(R)$ with $n \geq 2$: each of them is (finitely) ambiguous, but has a degree of ambiguity that grows monotonically with the exponent n . In particular, string b^3 has a degree of ambiguity equal to 4, and in general string b^n has a degree of ambiguity equal to 2^{n-1} with $\lim_{n \rightarrow \infty} 2^{n-1} = \infty$.

Yet expression R is not infinitely ambiguous, because there are no strings in the language $L(R)$ that have infinitely many possible derivations. In fact this happens only when an iterator (star or cross) has a nullable argument, which is not the case of R .

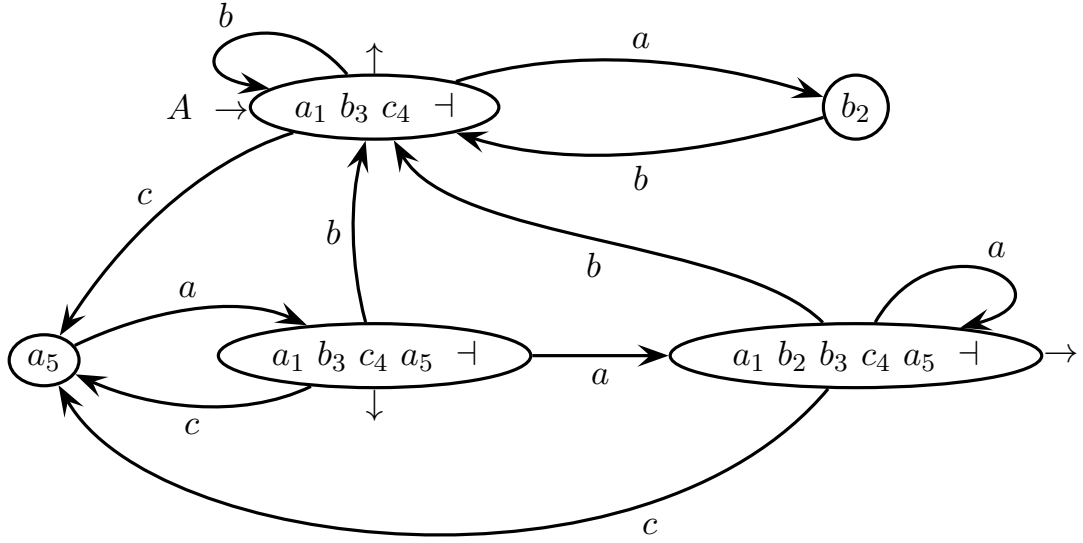
Note that expression R has (at least) another form of ambiguity: by numbering the letters as $(a_1 b_2 \mid b_3^+ \mid c_4 a_5^+)^*$, the valid string $caab \in L(R)$ can be derived from R as $c_4 a_5 a_1 b_2$ or $c_4 a_5 a_5 b_3$, thus with a degree of ambiguity equal to 2. The same holds for any valid string $ca a^n b$ with $n \geq 2$ (each string can be derived as $c_4 a_5 \dots a_5 a_5 b_3$ or $c_4 a_5 \dots a_5 a_1 b_2$), though all such strings have a constant degree of ambiguity equal to 2, regardless of their length.

- (c) Here the application of the Berry-Sethi (BS) method to the regular expression R :

$R_{\#} = (a_1 b_2 \mid b_3^+ \mid c_4 a_5^+)^* \dashv$	initials	$a_1 \ b_3 \ c_4 \ \dashv$
	terminal	followers
	a_1	b_2
	b_2	$a_1 \ b_3 \ c_4 \ \dashv$
	b_3	$a_1 \ b_3 \ c_4 \ \dashv$
	c_4	a_5
	a_5	$a_1 \ b_3 \ c_4 \ a_5 \ \dashv$

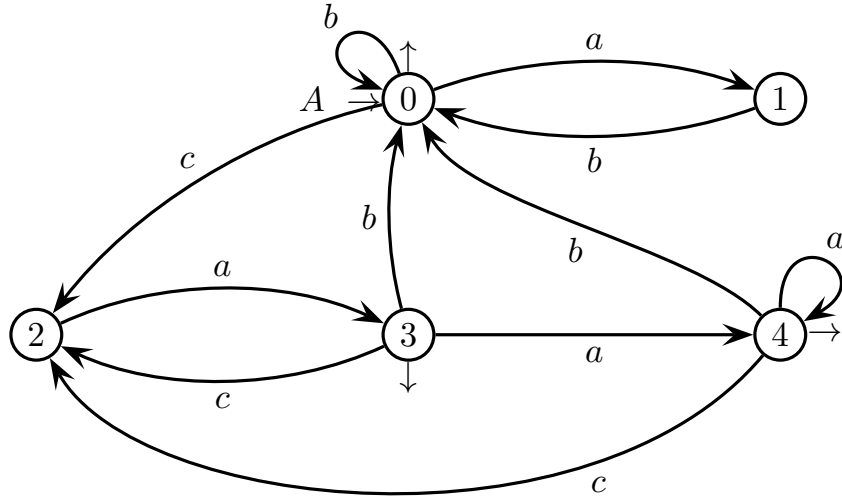
Notice that expression R is nullable, thus the initials of $R_{\#}$ must include the end-marker $\dashv$.

Here is the resulting deterministic automaton A , equivalent to expression R :



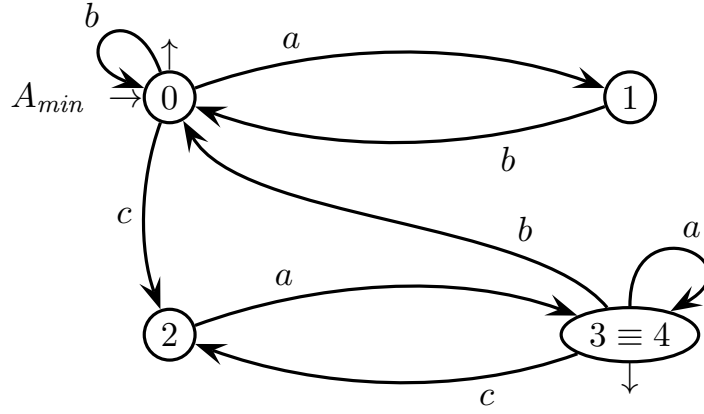
Automaton A has five states. It accepts the five valid strings of point (a) (the reader can list their accepting paths), while for instance it rejects the invalid strings a , aa , ba , and all those that end by letter c (none of which is generated by R). Therefore automaton A is reasonably correct.

(d) Here is the minimization of the deterministic automaton A . First state renaming:



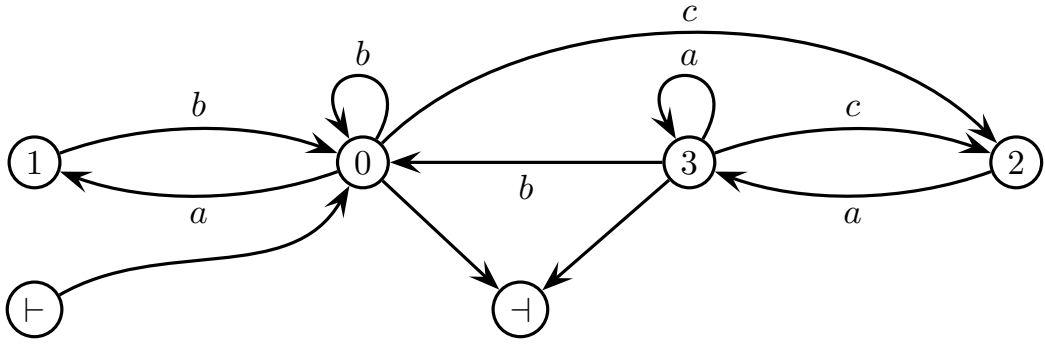
The two final states 3 and 4 of automaton A are undistinguishable: by input a both 3 and 4 stay inside their group, and by inputs b and c both go to the same states 0 (with b) and 2 (with c), respectively. Therefore states 3 and 4 can be merged. On the other hand, the two non-final states 1 and 2 are clearly distinguishable, because state 1 can only read input b and state 2 can only read input a . Eventually the final state 0 and the newly created final state $3 \equiv 4$ are distinguishable, because by input a state 0 goes to state 1, while state $3 \equiv 4$ loops back on itself.

Here is the resulting (deterministic) minimized automaton A_{min} :

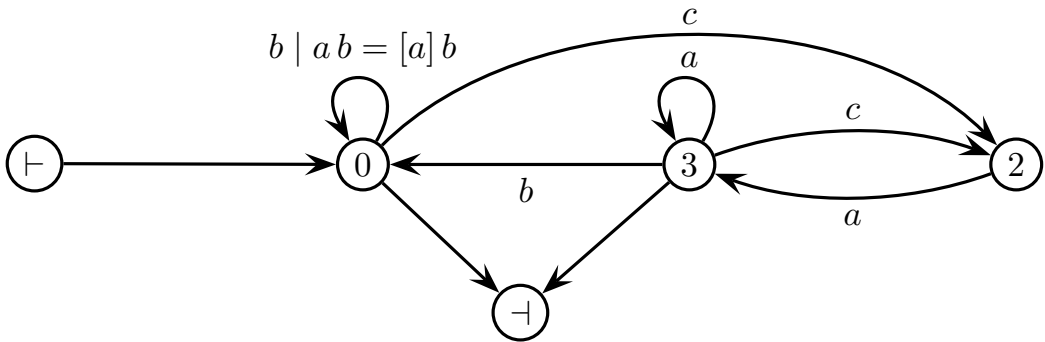


Automaton A_{min} is equivalent to automaton A , but has only four states.

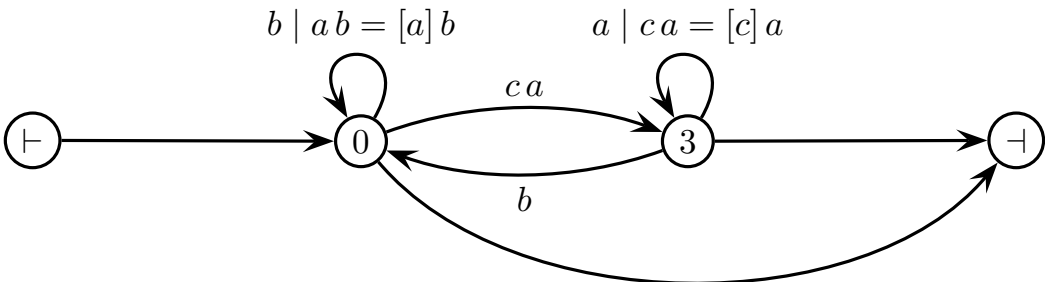
- (e) Here is the application of the node-elimination (BMC) method to the minimal automaton A_{min} , to obtain a regular expression R' equivalent to expression R . First a reshaping of the graph of A_{min} (the merged state $3 \equiv 4$ is renamed 3) and the normalization of initial and final states:



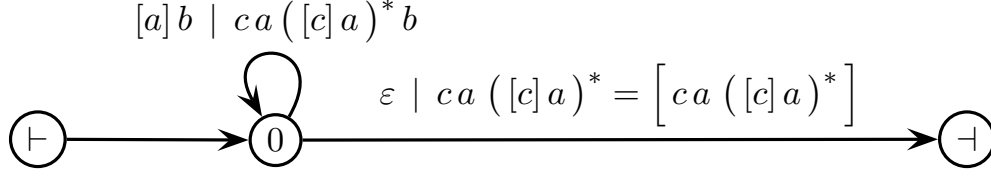
Next we can proceed to eliminate the inner nodes one by one. Eliminate node 1:



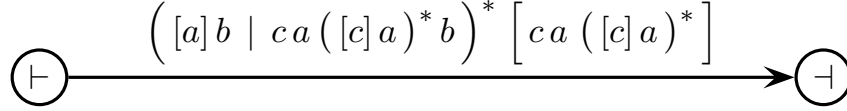
Eliminate node 2:



Eliminate node 3:



Finally eliminate node 0:



Therefore a possible formulation of expression R' , equivalent to R , is as follows:

$$R' = \left([a]b \mid ca([c]a)^*b \right)^* \left[ca([c]a)^* \right]$$

The regular expression R' is reasonably non-ambiguous: in R' each occurrence of the rightmost letter b is necessarily preceded by a letter c with one or more letters a in between, whereas the left occurrence of letter b may not be preceded by a letter c with only letters a in between. Thus the ambiguity mechanisms present in the original expression R , for the strings of type b^n and $ca a^n b$ with $n \geq 2$ (see point (b)), are broken in R' .

Just for completeness, we informally simplify expression R' . First, factor letter b on the right:

$$R' = \left(\left([a] \mid ca([c]a)^* \right) b \right)^* \left[ca([c]a)^* \right]$$

then, note that:

$$ca([c]a)^* = ca(ca \mid a)^* \equiv (ca^+)^+$$

hence, we have:

$$R' = \left(\left([a] \mid (ca^+)^+ \right) b \right)^* \left[(ca^+)^+ \right]$$

and finally:

$$R' = \left(\left([a] \mid (ca^+)^+ \right) b \right)^* (ca^+)^*$$

This reformulation of expression R' is also non-ambiguous (examine it by yourself), though it is rather more complex than the (equivalent) original ambiguous expression R .

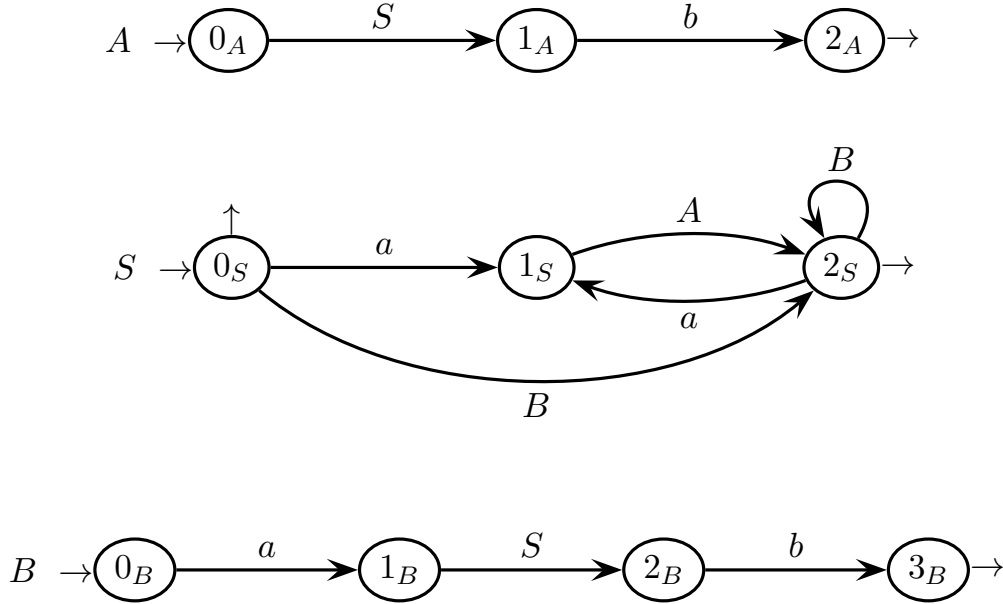
Even the original expression R can be informally simplified as follows, by first removing the internal cross iterator (in red) and then compacting the component $ab \mid b$ as $[a]b$:

$$R = (ab \mid b^+ \mid ca^+)^* \equiv (ab \mid b \mid ca^+)^* = ([a]b \mid ca^+)^*$$

This reformulation of expression R preserves equivalence and resolves the ambiguity of the strings of type b^n with $n \geq 2$, though the ambiguity of the strings of type $ca a^n b$ with $n \geq 2$ remains.

2 Syntax analysis and parsing methodologies (10 points)

Consider the grammar G represented by the machine net below, over the two-letter terminal alphabet $\Sigma = \{a, b\}$ and the three-symbol nonterminal alphabet $V = \{A, B, S\}$, with axiom machine S :



Answer the following questions (use the figures / tables / spaces on the next pages):

- (a) Draw the syntax trees for the valid sample string below:

$$a b a b \in L(G)$$

- (b) Draw the complete pilot graph of grammar G and determine if grammar G is ELR. If it is not ELR, explain what conflicts are present in the pilot.
- (c) Compute all the guide sets on the call arcs and exit arrows of the machine net of grammar G . Are there any guide set conflicts? Justify your answer.
- (d) Simulate the ELR(1) parsing process of the valid string below:

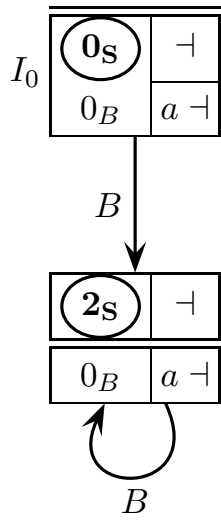
$$a a b b \in L(G)$$

During the simulation, in case a conflict situation is encountered, proceed as follows:

- if it is possible to both *shift* and *reduce*, always choose to *reduce*
- if it is possible to *reduce* a derivation in multiple ways, choose the one that removes the most items from the stack, if possible
- if the above rules are not sufficient to resolve the conflict, choose an arbitrary move amongst all valid possibilities

space for the tree(s) of the valid string $a b a b$ – question (a)

space for the complete pilot of grammar G and the ELR conflict analysis – question (b)



space for the guide sets of grammar G and the ELL conflict analysis – question (c)

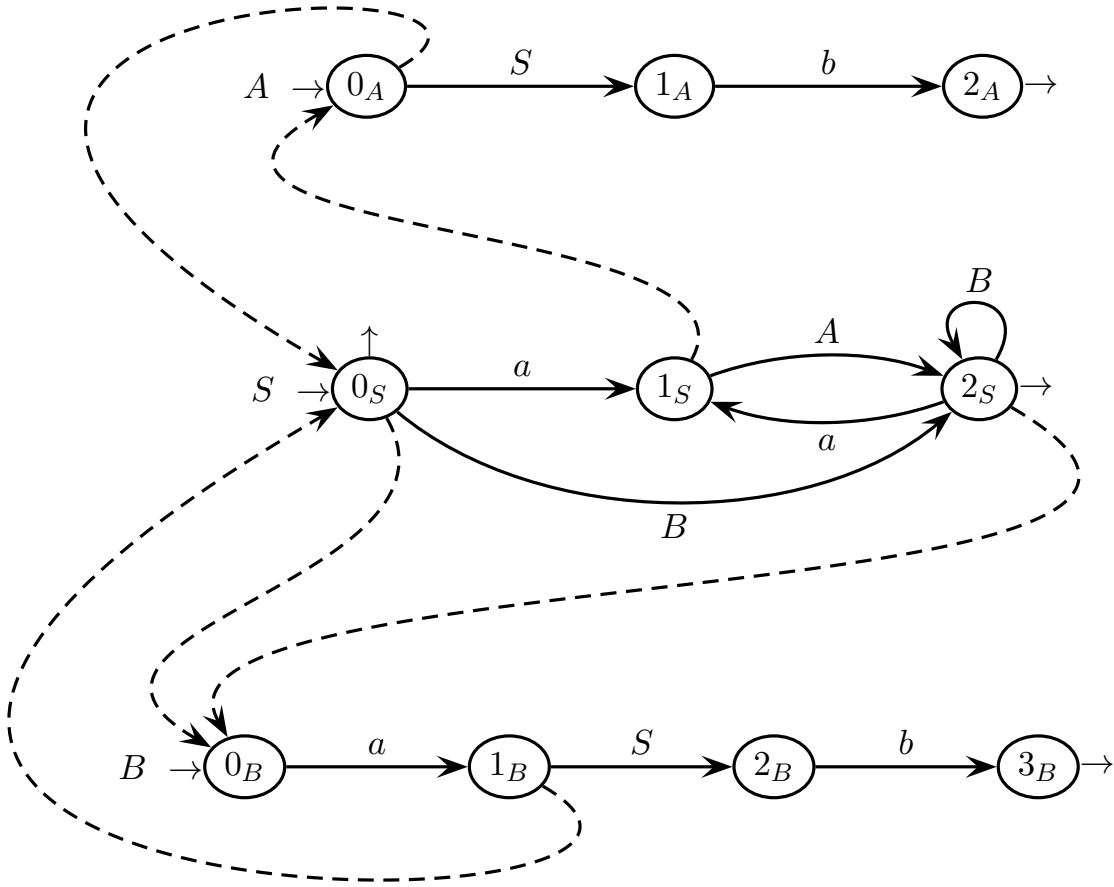


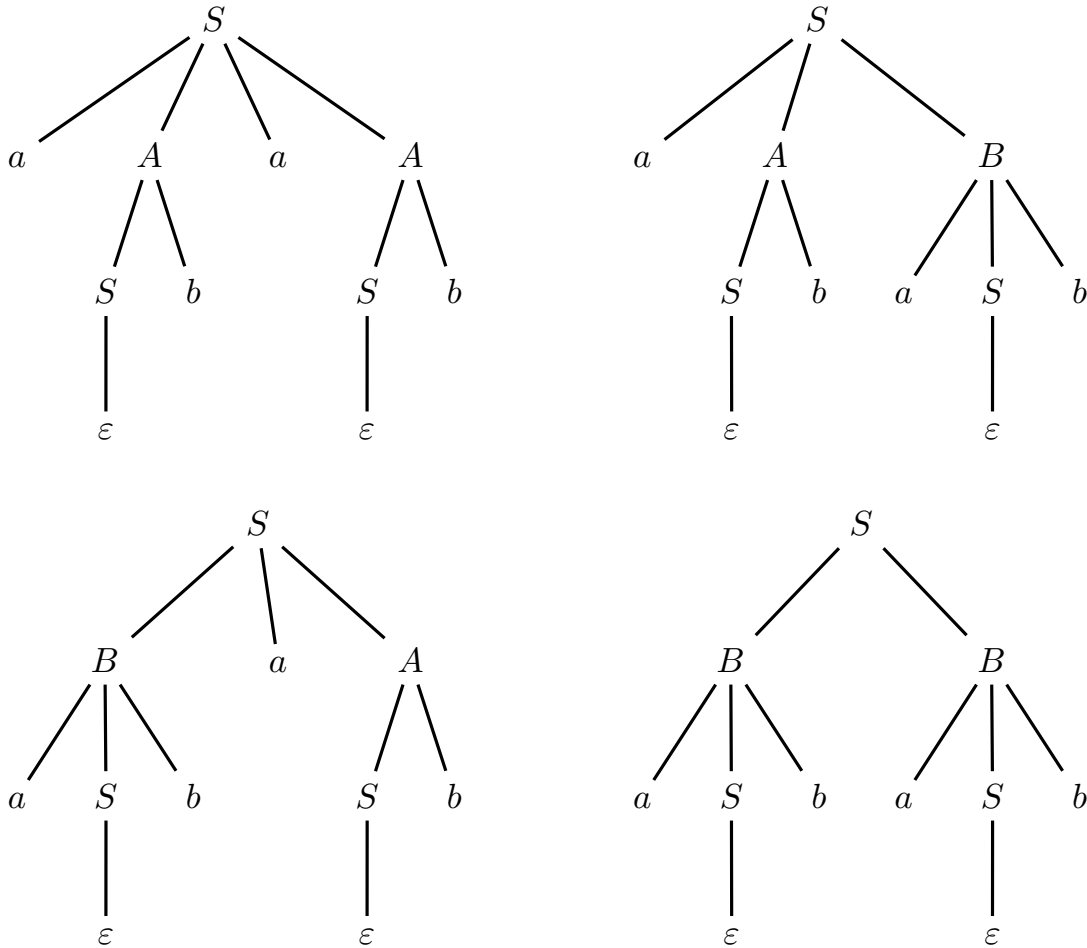
table for simulation of the ELR analysis of the valid string $a a b b$ – question (d)
(the number of rows is not significant)

[illegible]

Solution

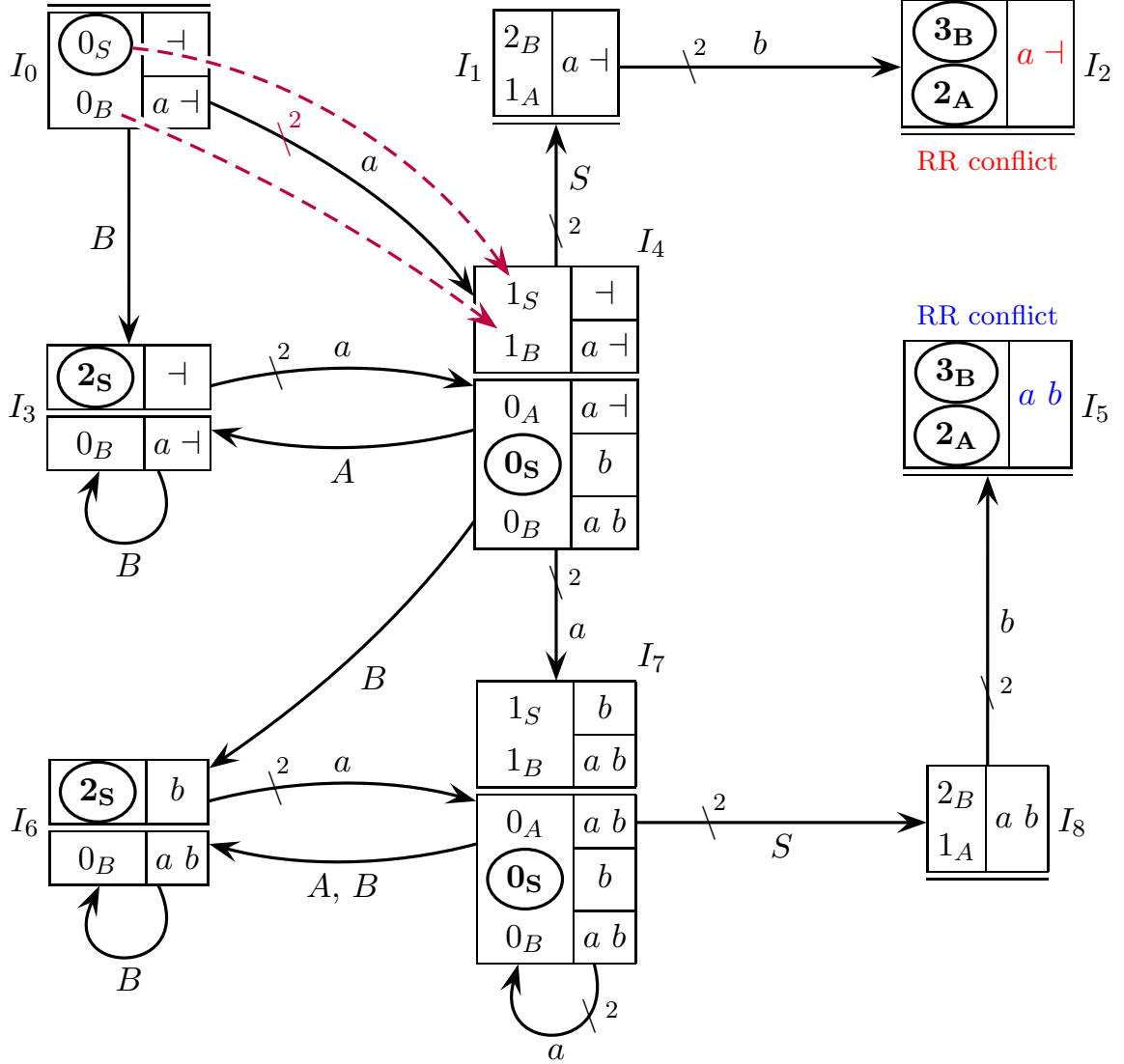
- (a) The language $L(G)$ is none other than the Dyck language, though generated by grammar G in an ambiguous way. In fact, each parenthesis pair ab can be independently obtained either from a derivation in the form $S \Rightarrow aA \Rightarrow aSb$ or from a derivation in the form $S \Rightarrow B \Rightarrow aSb$.

The example string $abab$ features two concatenated parenthesis pairs ab , which can be obtained in four possible ways, corresponding to all the combinations of the two ways in which each pair ab can be generated. Here are the four resulting syntax trees:



Notice that the degree of ambiguity grows with the number of parenthesis pairs, e.g., the valid string $ababab$ with three concatenated pairs is even more ambiguous, etc. In conclusion, grammar G is ambiguous and its degree of ambiguity is (at least) unlimited.

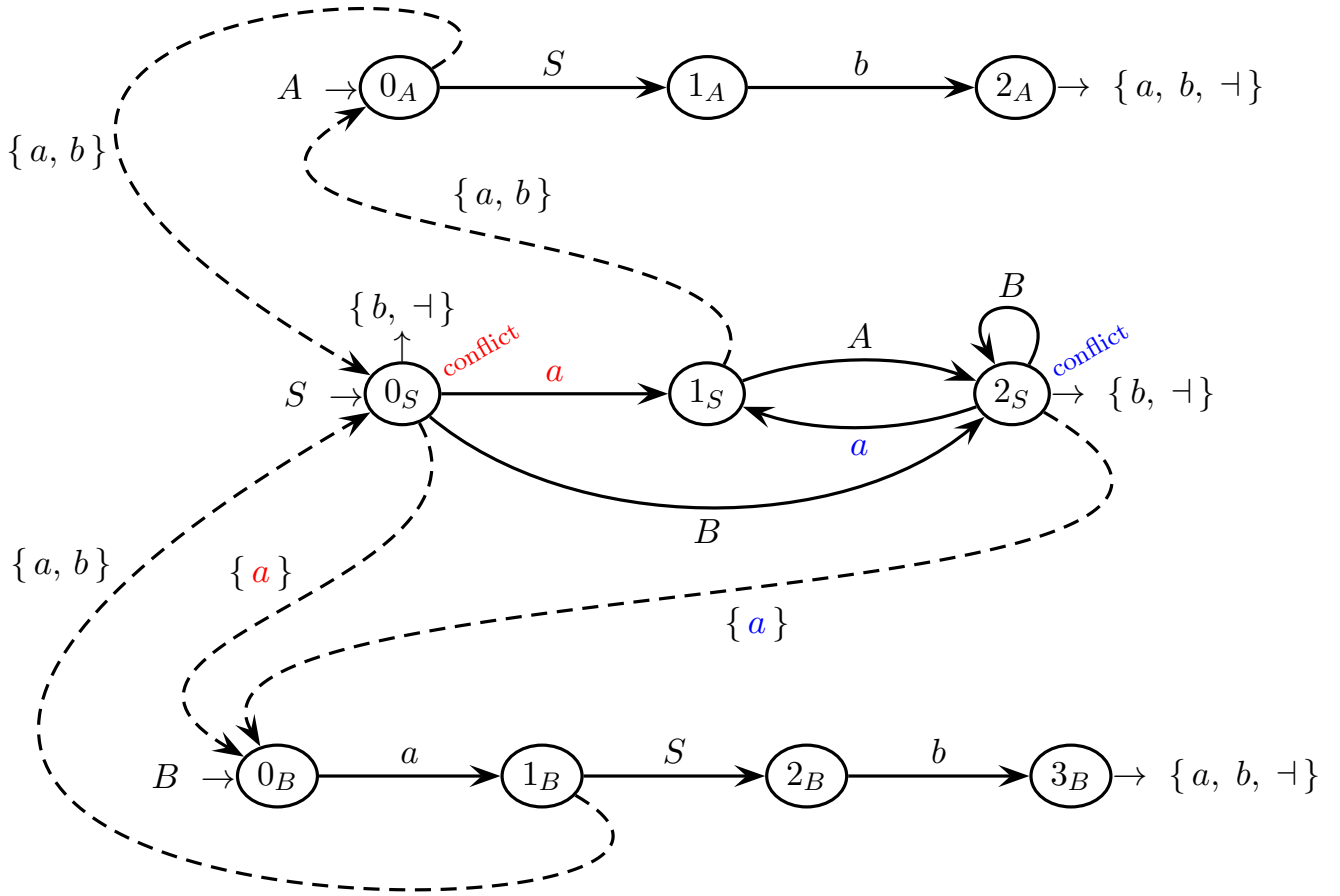
(b) Here is the complete pilot of grammar G , with just nine m-states:



The pilot is relatively simple. There are no shift-reduce (SR) conflicts. The pilot features several multiple (more precisely double) transitions (all those tagged with sign $\setminus^2$), e.g., $I_0 \xrightarrow{a} I_4$. Yet none of them is convergent, e.g., $I_0 \xrightarrow{a} I_4$ does not converge as it has the two different destination items 1_S and 1_B (see the dashed transitions $0_S \xrightarrow{a} 1_S$ and $0_B \xrightarrow{a} 1_B$); the same holds for all other double transitions. Thus *a-priori* there are no Convergence conflicts. Actually, there are only two reduce-reduce (RR) conflicts, in the m-states I_2 and I_5 , where the reduction candidates 3_B and 2_A share the same look-ahead terminals, namely $a \neg$ in I_2 and $a b$ in I_5 .

These two RR conflicts naturally emerge from the ambiguity between the two possible ways to derive each $a S b$ group. The m-state I_2 is reached if the ambiguous derivation appears at top level (not nested), while the m-state I_5 is reached for nested groups. In conclusion, grammar G is not ELR, which was expected since it is even ambiguous.

(c) Here are all the guide sets on the call arcs and exit arrows of the machine net of grammar G :

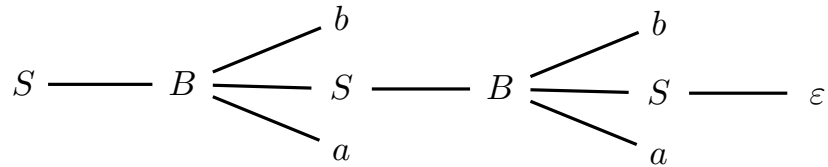


Of course grammar G cannot be ELL(1), as it is not ELR(1) either. Thus guide set conflicts are expected to show up somewhere. All conflicts appear in the net states directly related to the ambiguous derivations we already observed for string $a b a b$, namely states 0_S and 2_S for pairs $a b$ and $a b$, respectively, in both cases on terminal a . This confirms that grammar G is not ELL.

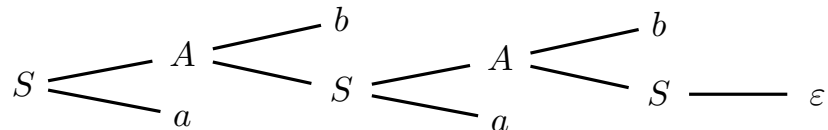
(d) Here is the simulation of the ELR(1) parsing process of the valid sample string $a a b b \in L(G)$:

<i>stack of m-states and grammar symbols</i>	<i>input left to scan</i>	<i>move executed</i>	<i>note</i>
I_0	$a a b b \dashv$	initial configuration	
$I_0 a I_4$	$a b b \dashv$	terminal shift: $I_0 \xrightarrow{a} I_4$	
$I_0 a I_4 a I_7$	$b b \dashv$	terminal shift: $I_4 \xrightarrow{a} I_7$	
$I_0 a I_4 a I_7$	$b b \dashv$	reduction: $\varepsilon \rightsquigarrow S$	
$I_0 a I_4 a I_7 S I_8$	$b b \dashv$	nonterminal shift: $I_7 \xrightarrow{S} I_8$	
$I_0 a I_4 a I_7 S I_8 b I_5$	$b \dashv$	terminal shift: $I_8 \xrightarrow{b} I_5$	
$I_0 a I_4$	$b \dashv$	reduction: $a S b \rightsquigarrow B$	†
$I_0 a I_4 B I_6$	$b \dashv$	nonterminal shift: $I_4 \xrightarrow{B} I_6$	
$I_0 a I_4$	$b \dashv$	reduction: $B \rightsquigarrow S$	
$I_0 a I_4 S I_1$	$b \dashv$	nonterminal shift: $I_4 \xrightarrow{S} I_1$	
$I_0 a I_4 S I_1 b I_2$	$\dashv$	terminal shift: $I_1 \xrightarrow{b} I_2$	
I_0	$\dashv$	reduction: $a S b \rightsquigarrow B$	†
$I_0 B I_3$	$\dashv$	nonterminal shift: $I_0 \xrightarrow{B} I_3$	
I_0	$\dashv$	reduction: $B \rightsquigarrow S$	
initial stack	empty input	last move reduction to axiom	
acceptance condition verified			

Note that in the moves marked with † it is possible to reduce either $a S b \rightsquigarrow B$ or $S b \rightsquigarrow A$, due to a reduce-reduce conflict situation (see point (b)). However, the disambiguation rules force us to discard the latter option, namely $S b \rightsquigarrow A$, as it would remove fewer items from the stack compared with the former (two items vs three). In effect, this results in the same moves obtainable by discarding machine A and all its related transitions from the machine net. Here is the resulting syntax tree (drawn from left to right):



Just for completeness, note that also string $a a b b$ is ambiguous. By making different reduction choices (those discarded before like $S b \rightsquigarrow A$), more trees could be obtained, like for instance:



and also other trees, by combining the possible reductions in all ways.

3 Grammar design – translation – semantic analysis (5 + 5 points)

1. We want to model the declaration of *generic types* in a language similar to, e.g., C#. Such declarations appear as in the following examples:

```
class Box <T, U> {...}
```

```
class Repo <T> where T : BaseEntity, IDisposable, IAsyncDisposable, new ( ) {...}
```

```
class PairMap <TKey, TValue>  
    where TKey    : notnull, IComparable  
    where TValue  : class  
{...}
```

```
struct S <T> where T : struct {...}
```

More in detail, a single declaration of a *generic*:

- is introduced by the keyword **struct** or **class**
- has a name, i.e., an identifier, followed by the declaration(s) of one or more type parameter(s), i.e., identifiers, surrounded by a pair of angular brackets “< >”
- has zero or more *constraint clauses* that apply to the type parameters
- and has a block between braces “{ }”, the content of which is modeled by a nonterminal BLOCK, not to be further expanded

Identifiers are schematized by terminal **id**, not to be further expanded.

A constraint clause is composed of the keyword **where**, followed by (i) an identifier, (ii) a colon “:”, and (iii) one or more constraints, separated by commas “,”. A constraint is classified as follows:

- primary constraint: either an identifier, or one of the terminals **class**, **struct** or **unmanaged**
- secondary constraint: either an identifier or the terminal **notnull**
- constructor constraint: **new ()**

If a primary constraint appears, it occurs in the first position of the list of constraints. If a constructor constraint appears, it occurs in the last position. Only secondary constraints can appear more than once.

Answer the following questions (use the figures / tables / spaces on the next pages):

- Write an EBNF grammar (possibly ambiguous) that covers the declaration of a single *generic* as specified above.
 - Draw the syntax tree for the **PairMap** declaration example above.
 - Consider the case where an additional limitation is introduced: the primary constraints **struct** and **unmanaged** are incompatible with the constructor constraint **new ()**. Modify the grammar so as to handle this case as well.
-

Solution

(a) Here is a viable EBNF (slightly ambiguous) grammar that generates *generics* (axiom GEN):

$$\left\{ \begin{array}{l} \langle \text{GEN} \rangle \rightarrow (\text{class} \mid \text{struct}) \text{id} \langle \text{LPAR} \rangle \langle \text{CLA} \rangle^* \text{"{"} \langle \text{BLOCK} \rangle \text{"}" } \\ \langle \text{LPAR} \rangle \rightarrow \text{"<" id (" , " id)^* ">" } \\ \langle \text{CLA} \rangle \rightarrow \text{where id ":"} \langle \text{CON} \rangle \\ \langle \text{CON} \rangle \rightarrow \left(\left(\langle \text{PRI} \rangle [\text{" , " } \langle \text{LSEC} \rangle] \mid \langle \text{LSEC} \rangle \right) [\text{" , " } \langle \text{CNS} \rangle] \right) \mid \langle \text{CNS} \rangle \\ \langle \text{PRI} \rangle \rightarrow \text{id} \mid \text{class} \mid \text{struct} \mid \text{unmanaged} \\ \langle \text{LSEC} \rangle \rightarrow \langle \text{SEC} \rangle (\text{" , " } \langle \text{SEC} \rangle)^* \\ \langle \text{SEC} \rangle \rightarrow \text{id} \mid \text{nonnull} \\ \langle \text{CNS} \rangle \rightarrow \text{new " (" " ")"} \end{array} \right.$$

The main difficulty in formulating the grammar lies in the list of constraints, which must be correctly separated by commas (without extra or missing commas in any case), regardless of the item types contained in the list, which are themselves restricted. In the solution we propose, we introduce a nonterminal LSEC that expands to a comma-separated list of one or more secondary constraints. With this definition, the rule CON representing the full list is a (still comma-separated) sequence of up to three elements — PRI, LSEC and CNS — where any item might be missing (but not all three) and all items must appear in this order.

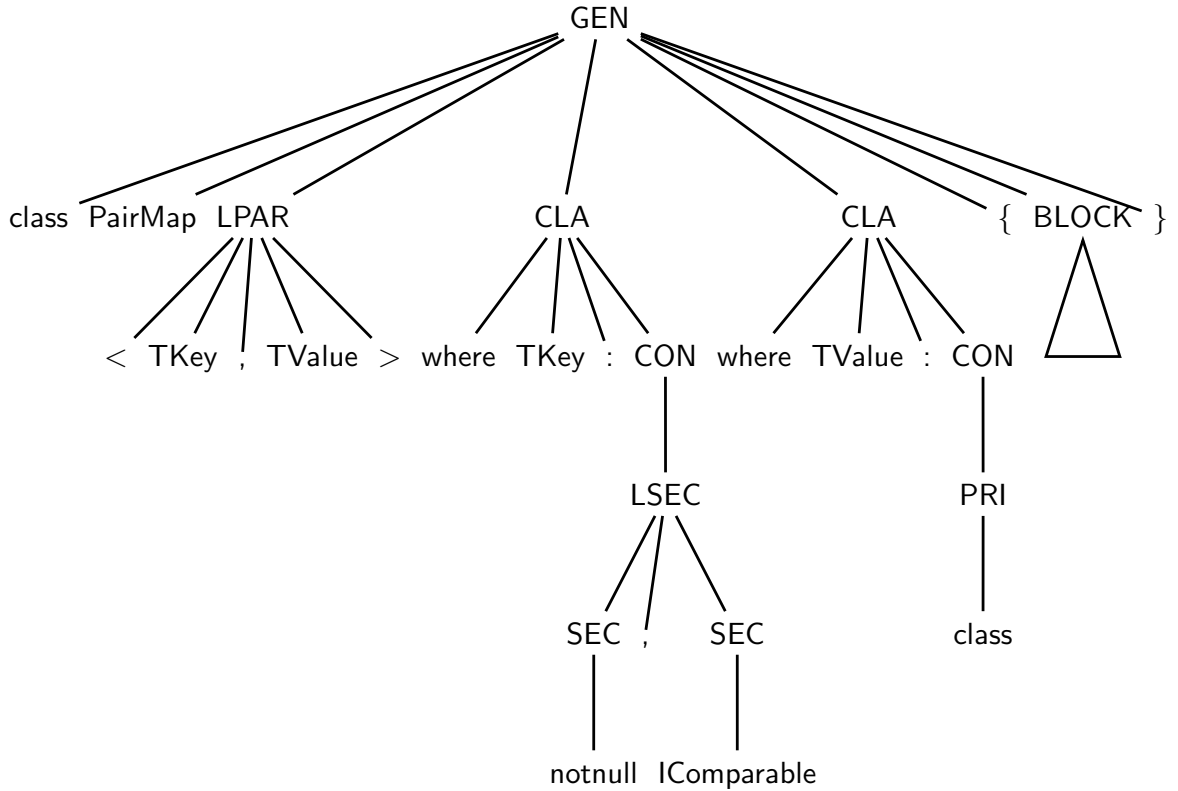
Notice that in such a list (like in any other list with separators) the only element that might not be preceded by a comma, is the first one. Hence we can distinguish three cases, depending on the first item:

- if the first item is $\langle \text{PRI} \rangle$, the rule shall expand to $\langle \text{PRI} \rangle [\text{" , " } \langle \text{LSEC} \rangle] [\text{" , " } \langle \text{CNS} \rangle]$
- if the first item is $\langle \text{LSEC} \rangle$, the rule shall expand to $\langle \text{LSEC} \rangle [\text{" , " } \langle \text{CNS} \rangle]$
- if the first item is $\langle \text{CNS} \rangle$, the rule shall expand to $\langle \text{CNS} \rangle$

The three cases can be simply joined with a union to obtain a viable expression for the right-hand-side of nonterminal CON. One can apply simple factorizations to obtain the rule above.

Since both nonterminals PRI and SEC can generate an identifier, i.e., terminal id, and since they appear united in the rule of CON, the grammar is ambiguous (union ambiguity). This results from the choice to rigorously model the language specification, which explicitly states that an id can originate from both PRI and SEC. Note however that if nonterminal PRI were disallowed to generate an identifier, a (weakly) equivalent non-ambiguous grammar would be obtained. The language generated would not change, but from a structural point of view the grammar would no longer be rigorously conforming to the description given in the exercise text.

(b) Here is the syntax tree for the `PairMap` declaration example (`BLOCK` is left unexpanded):



Each occurrence of terminal `id` is replaced by its actual expansion in the `PairMap` example.

(c) Here is a viable modified grammar (the new rules and the modified rule parts are in red):

$$\left\{ \begin{array}{l}
 \langle \text{GEN} \rangle \rightarrow (\text{class} \mid \text{struct}) \text{id} \langle \text{LPAR} \rangle \langle \text{CLA} \rangle^* \text{"\{"} \langle \text{BLOCK} \rangle \text{"\}" } \\
 \langle \text{LPAR} \rangle \rightarrow \text{"<"} \text{id} (\text{" , " id})^* \text{">} \\
 \langle \text{CLA} \rangle \rightarrow \text{where id " : " } \langle \text{CON} \rangle \\
 \langle \text{CON} \rangle \rightarrow \left(\left(\langle \text{PRI} \rangle [\text{" , " } \langle \text{LSEC} \rangle] \mid \langle \text{LSEC} \rangle \right) [\text{" , " } \langle \text{CNS} \rangle] \right) \\
 \quad \mid \langle \text{NO_CNS} \rangle [\text{" , " } \langle \text{LSEC} \rangle] \mid \langle \text{CNS} \rangle \\
 \langle \text{PRI} \rangle \rightarrow \text{id} \mid \text{class} \mid \text{struct} \mid \text{unmanaged} \\
 \langle \text{NO_CNS} \rangle \rightarrow \text{struct} \mid \text{unmanaged} \\
 \langle \text{LSEC} \rangle \rightarrow \langle \text{SEC} \rangle (\text{" , " } \langle \text{SEC} \rangle)^* \\
 \langle \text{SEC} \rangle \rightarrow \text{id} \mid \text{nonnull} \\
 \langle \text{CNS} \rangle \rightarrow \text{new " (" ") " }
 \end{array} \right.$$

The modified grammar separates the primary constraints of type `struct` and `unmanaged` from the constructor constraint `new ( )`, by means of an additional nonterminal `NO_CNS`. Therefore a new (fourth) case is added to the ones already listed at point (a) in the rule of nonterminal `CON`: if the first element of the list is the nonterminal `NO_CNS`, then the rest of the list only consists of secondary constraints, with no constructor constraint at the end.

The (limited) ambiguity form of the original grammar (see point (a)) is still present in the modified one. For the `PairMap` declaration example, the modified grammar produces a syntax tree identical to the tree produced by the original grammar (see point (b)).

2. Consider the BNF grammar G below, which models a simplified programming language (axiom S):

$$G \left\{ \begin{array}{ll} S \rightarrow I & \\ I \rightarrow L I \mid C I \mid O I \mid \varepsilon & // \text{ list of statements} \\ L \rightarrow l \text{ id} & // \text{ label statement, declares a label id} \\ C \rightarrow c \text{ id} & // \text{ comefrom statement, uses a label id} \\ O \rightarrow o & // \text{ any other statements} \end{array} \right.$$

where:

- nonterminal I represents the program as a list of zero or more statements
- nonterminal L represents a label statement, composed of the keyword **label** (schematized by terminal l for brevity) and terminal **id**, an identifier representing the label declared in the statement
- nonterminal C represents a comefrom statement, composed of the keyword **comefrom** (schematized by terminal c for brevity) and terminal **id**, an identifier representing the label used in the statement
- nonterminal O represents any other statement, here schematized as a single terminal o to emphasize that no instances of nonterminal L or C may appear in O

To ensure the semantic correctness of a program in the proposed language, we need to ensure that:

- no two L statements declare the same label
- for each C statement, the label used is declared somewhere, no matter whether before or after C
- no two C statements use the same label, in order to avoid nondeterminism

To this purpose, build an attribute grammar, based on G , that employs the following attributes:

<i>name</i>	<i>domain</i>	<i>type</i>	<i>non-term(s)</i>	<i>meaning</i>
d	multi-set of labels	left	$S I L$	labels declared in the label statements
u	multi-set of labels	left	$S I C$	labels used in the comefrom statements
md	set of labels	left	S	labels with multiple declarations
mu	set of labels	left	S	labels with multiple uses
wd	set of labels	left	S	labels used without declaration

We recall that a *multi-set* is a set that can contain repeated elements. Given a multi-set m and a possible element e , the notation $|m|_e \geq 0$ indicates the *number of occurrences* of the element e in the multi-set m , i.e., 0 (no occurrence), 1 (one occurrence), 2 (two occurrences), etc. We also assume that function $n()$, without arguments, returns the semantic value associated with terminal **id**.

Answer the following questions (use the figures / tables / spaces on the next pages):

- Write the semantic functions for each rule.
- Compute the attributes for the string below (letters a and b are labels, i.e., instances of **id**):

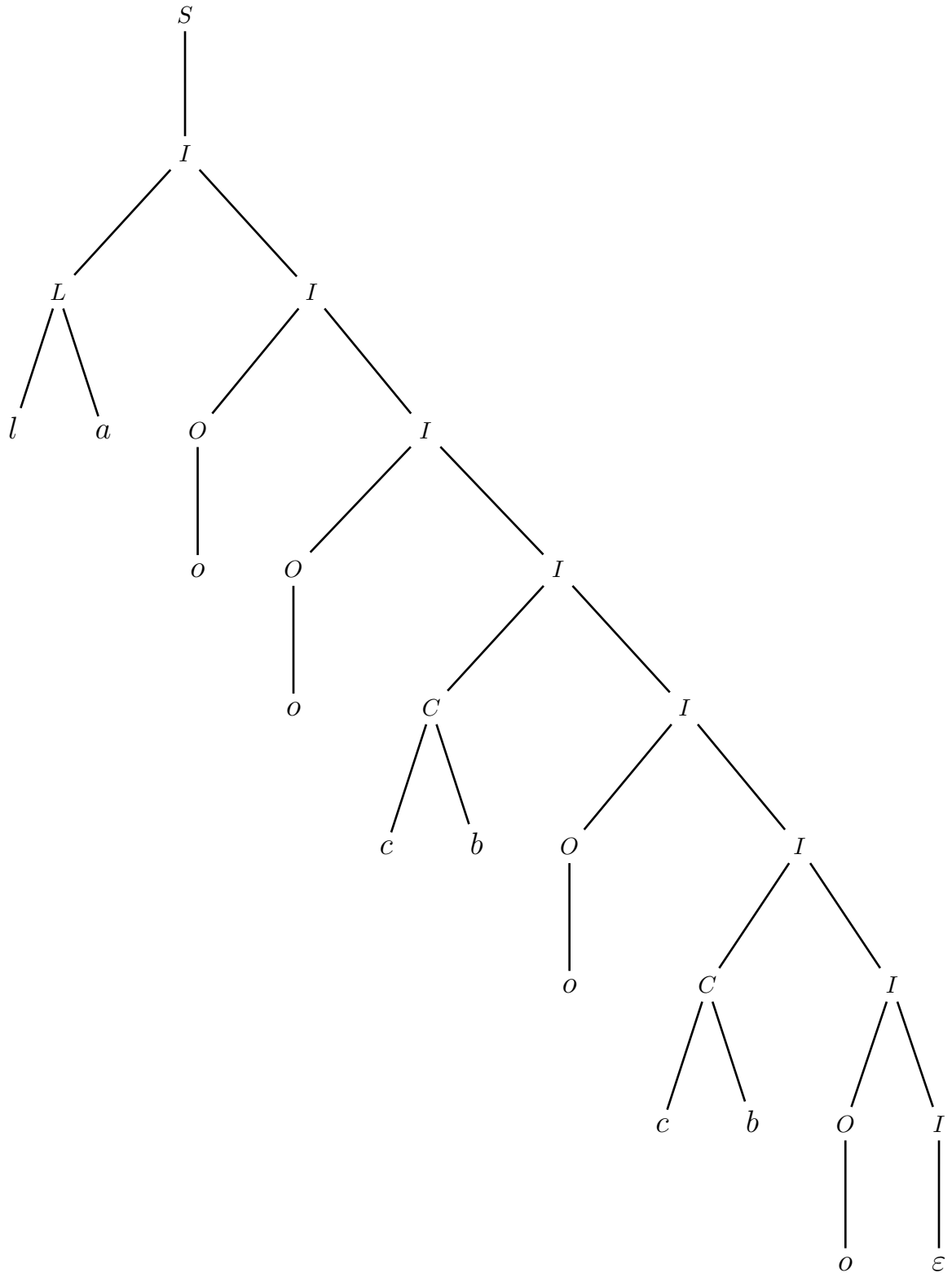
$$l a o o c b o c b o \in L(G)$$

- Write the integrated syntactic-semantic recursive-descent procedure of nonterminal I .
- Is it possible to design a different attribute grammar that uses only attributes with domain *set of labels*, i.e., without repeated elements? If so, would it still be possible to use only synthesized attributes?

write here the semantic functions of the attribute grammar – question (a)

#	<i>syntax</i>	<i>semantics</i>
1:	$S_0 \rightarrow I_1$	
2:	$I_0 \rightarrow L_1 I_2$	
3:	$I_0 \rightarrow C_1 I_2$	
4:	$I_0 \rightarrow O_1 I_2$	
5:	$I_0 \rightarrow \varepsilon$	
6:	$L_0 \rightarrow l \text{ id}$	
7:	$C_0 \rightarrow c \text{ id}$	
8:	$O_0 \rightarrow o$	// nonterminal O does not have any attributes

decorate with the attributes the tree of the string $laocbocbo$ – question (b)



write the integrated syntactic-semantic recursive-descent procedure of nonterminal I – question (c)

space for answering question (d)

Solution

(a) Here is the attribute grammar with the given attributes (all of them are synthesized):

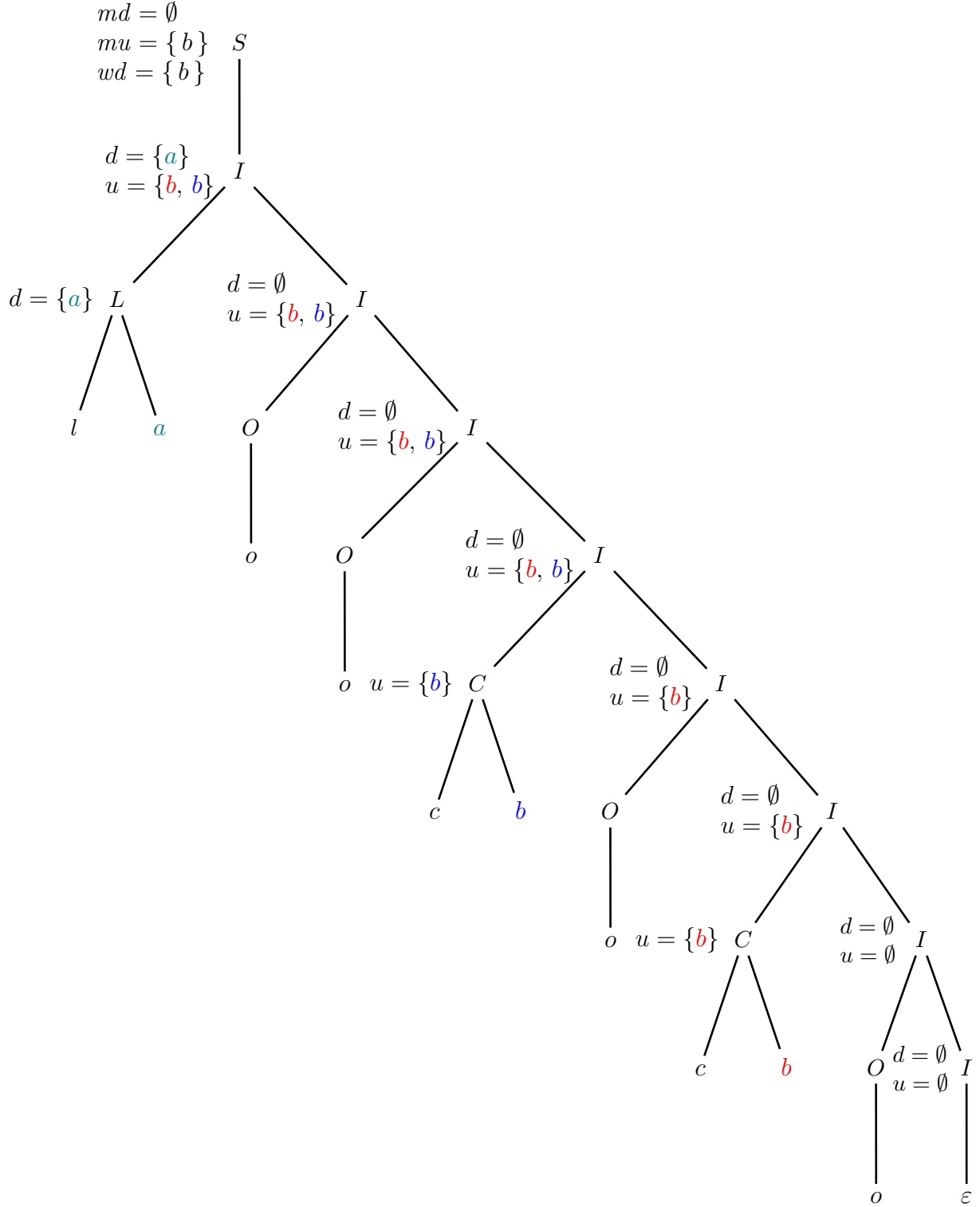
#	<i>syntax</i>	<i>semantics</i>	<i>comment</i>
1:	$S_0 \rightarrow I_1$	$md_0 := \{e \mid d_1 _e > 1\}$ // elements e declared more than once $mu_0 := \{e \mid u_1 _e > 1\}$ // elements e used more than once $wd_0 := \{e \mid e \notin d_1 \wedge e \in u_1\}$ // elements e undeclared but used	
2:	$I_0 \rightarrow L_1 I_2$	$d_0 := d_1 \uplus d_2$ // disjoint union of multi-sets $u_0 := u_2$ // send up attribute	
3:	$I_0 \rightarrow C_1 I_2$	$d_0 := d_2$ // send up attribute $u_0 := u_1 \uplus u_2$ // disjoint union of multi-sets	
4:	$I_0 \rightarrow O_1 I_2$	$d_0 := d_2$ // send up attribute $u_0 := u_2$ // send up attribute	
5:	$I_0 \rightarrow \varepsilon$	$d_0 := \emptyset$ // initialize attribute $u_0 := \emptyset$ // initialize attribute	
6:	$L_0 \rightarrow l \text{ id}$	$d_0 := \{n(\)\}$ // pick up declared label	
7:	$C_0 \rightarrow c \text{ id}$	$u_0 := \{n(\)\}$ // pick up used label	
8:	$O_0 \rightarrow o$	// nonterminal O does not have any attributes	

Symbol $\uplus$ denotes the union of multi-sets (*disjoint union*), which arithmetically adds the multiplicity of the elements, e.g., $\{e\} \uplus \{e\} = \{e, e\}$. Multi-sets are sometimes denoted, e.g., $[e, e]$, with square brackets instead of curly ones (here we use the curly ones however).

Rules 6 and 7 create two singleton multi-sets d and u containing the locally declared and used label, respectively (with multiplicity one). Rules 2 (add a declaration), 3 (add a usage), 4 (send up each multi-set d and u as-is) and 5 (initialization) update the multi-sets, so collecting all the declared and used labels with their multiplicity. Rule 1 eventually computes the (standard) sets of multiple declared labels, multiple used ones and undeclared ones.

The attribute grammar is acyclic, as all the attribute dependencies are always bottom-up (from child nodes to parent node), thus it is computable.

(b) Here are the attribute values in all the tree nodes for the string $l a o o c b o c b o$:



In conclusion, in the root node we have these sets:

$$md = \emptyset \quad mu = \{b\} \quad wd = \{b\}$$

Since label b appears in the set mu , it is multiple used (in particular twice), and since it appears in the set wd , it is undeclared. Therefore the string is semantically incorrect.

Note that here attribute u really works as a multi-set, since it contains the used label b twice. Attribute d is also a multi-set, though in this specific example it works as a standard set because there are no multiple declared labels. Attributes md , mu and wd are standard sets by definition.

- (c) The attribute grammar is one-sweep, as it only has left attributes. The guide sets (with $k = 1$) of each alternative of nonterminal I , and of the rules of nonterminals L , C , O and S , are these:

$$\begin{array}{ll}
 S \rightarrow I & \{l, c, o, \vdash\} \\
 I \rightarrow L I & \{l\} \\
 I \rightarrow C I & \{c\} \\
 I \rightarrow O I & \{o\} \\
 I \rightarrow \varepsilon & \{\vdash\} \\
 L \rightarrow l \text{ id} & \{l\} \\
 C \rightarrow c \text{ id} & \{c\} \\
 O \rightarrow o & \{o\}
 \end{array}$$

All alternative guide sets are disjoint. Thus the syntactic support G is even LL(1). Therefore the attribute grammar has an integrated syntactic-semantic recursive-descent analyzer.

Here is the (pseudo-code) procedure for nonterminal I , in LL style, without (left) and with (right) optimizations, and the procedure headers of L and C on top (O is parameterless):

<pre> procedure L (out: d_0) procedure I (out: d_0, u_0) local vars d_1, d_2, u_1, u_2 // multi-set if $cc = l$ then // $I \rightarrow L I$ call L (d_1) if $cc \in \{l, c, o, \vdash\}$ then call I (d_2, u_2) if $cc = \vdash$ then $d_0 := d_1 \uplus d_2$ $u_0 := u_2$ return else error else error else if $cc = c$ then // $I \rightarrow C I$ call C (u_1) if $cc \in \{l, c, o, \vdash\}$ then call I (d_2, u_2) if $cc = \vdash$ then $d_0 := d_2$ $u_0 := u_1 \uplus u_2$ return else error else error else if $cc = o$ then // $I \rightarrow O I$ call O () if $cc \in \{l, c, o, \vdash\}$ then call I (d_2, u_2) $d_0 := d_2$ $u_0 := u_2$ return else error else if $cc = \vdash$ then // $I \rightarrow \varepsilon$ $d_0 := \emptyset$ $u_0 := \emptyset$ return else error end procedure </pre>	<pre> procedure C (out: u_0) // optimized version procedure I (out: d_0, u_0) local vars d_1, d_2, u_1, u_2 // multi-set if $cc = l$ then // $I \rightarrow L I$ call L (d_1) if $cc \in \{l, c, o, \vdash\}$ then call I (d_2, u_0) if $cc = \vdash$ then $d_0 := d_1 \uplus d_2$ return else if $cc = c$ then // $I \rightarrow C I$ call C (u_1) if $cc \in \{l, c, o, \vdash\}$ then call I (d_0, u_2) if $cc = \vdash$ then $u_0 := u_1 \uplus u_2$ return else if $cc = o$ then // $I \rightarrow O I$ call O () if $cc \in \{l, c, o, \vdash\}$ then call I (d_0, u_0) return else if $cc = \vdash$ then // $I \rightarrow \varepsilon$ $d_0 := \emptyset$ $u_0 := \emptyset$ return error // all error cases end procedure </pre>
---	--

The reader might finalize the analyzer and write procedures for nonterminals L , C , O and S .

Here is again the procedure for nonterminal I , written in Python and with more advanced code optimization (for Python multi-sets must be enclosed in square brackets):

```
def I():
    // all vars decls, uses, d and u are multi-sets of labels
    decls, uses = [], [] // initialize label multi-sets
    if cur == 'l' :      // alternative rule  $I \rightarrow LI$ 
        decls += L()    // pick up declared label
    elif cur == 'c' :    // alternative rule  $I \rightarrow CI$ 
        uses += C()     // pick up used label
    elif cur == 'o' :    // alternative rule  $I \rightarrow OI$ 
        O()             // other statement (irrelevant)
    elif cur == '⊥' :    // alternative  $I \rightarrow \varepsilon$  (program end)
        return [], []   // no labels to return
    else:                // parsing error
        error           // print diagnostic and stop
    d, u = I()           // label(s) returned by  $I$ 
    decls += d           // add declared label(s)
    uses += u           // add used label(s)
    return decls, uses   // return label multi-sets
```

The call to procedure I , which is common to the three alternative rules $I \rightarrow LI$, $I \rightarrow CI$ and $I \rightarrow OI$, is factored after the concatenated conditional structure *if-elif-else*. This looks like putting the rule of nonterminal I in EBNF form, i.e., $I \rightarrow (L \mid C \mid O) I \mid \varepsilon$. Consequently the definition and computation of the local variables have to be adjusted. The same factorization could be applied to the previous pseudo-code versions (also with adjusting the local variables). The above version in Python may be shorter and more efficient than the pseudo-code versions, as it saves a few operations, yet it is less adherent to how the grammar models and computes the attributes. For instance, the multi-sets `decls` and `uses` do not have an exact correspondence with the grammar attributes. The Python version is shown here just for completeness.

- (d) To eliminate the multi-sets and use only standard sets, that is, to redefine the attributes d and u as sets *without multiplicity*, there are (at least) *two* possible approaches, sketched below:
- change to *right* (inherited) the type of the attributes d and u , and use them (as standard sets) to collect in *top-down* way the labels declared and used across the tree, respectively
 - keep to *left* (synthesized) the type of the attributes d and u , and use them (as standard sets) to collect in *bottom-up* way the labels declared and used across the tree, respectively

In both cases, the attributes md and mu are still standard sets and remain of type *left*, but must be *associated* also with nonterminal I , besides nonterminal S . Such attributes can be computed bottom-up in all nodes I of the tree, up to and including the root node S , as follows. If a label id is locally declared (or used) by the nonterminal L (or C) in a tree node of type $I \rightarrow LI$ (or $I \rightarrow CI$), and if the same id already appears in the set d (or u), then id is multiple declared (or used) and must be put in the set md (or mu). The set wd remains unchanged.

In conclusion, multi-sets are unnecessary (though useful) and standard sets are sufficient for the attribute grammar. This change of data-structures can be done with either right attributes or only left attributes. The reader may wish to further develop and encode such approaches.

4 Bonus question (5 points)

Given a three-letter terminal alphabet $\Sigma = \{a, b, c\}$, consider the three languages L_1 , L_2 and L_3 below, all over the alphabet Σ :

- language L_1 is defined by the following characteristic predicate:

$$L_1 = \{ a^n b^m c^{m-n} \mid m > n > 0 \}$$

- language L_2 is generated by the following BNF grammar G_2 (axiom S):

$$G_2 \left\{ \begin{array}{l} S \rightarrow a S U \mid T \\ U \rightarrow b S \\ T \rightarrow b T V \mid \varepsilon \\ V \rightarrow c T \end{array} \right.$$

- language L_3 is defined by the following characteristic predicate:

$$L_3 = \{ a^n b^m c^l \mid m > 0, n > 1, l > 0 \}$$

Answer the following questions (use the figures / tables / spaces on the next pages):

- What are the smallest language classes to which each of the three languages belongs ?
 - Construct a recognizer automaton for the intersection language $L_2 \cap L_3$.
 - Construct a recognizer automaton for language L_1 .
-

Solution

(a) The smallest language family for both languages L_1 and L_2 is the context-free one (CF), while that for language L_3 is the regular one (REG). Here are the arguments to prove so:

- Language L_1 consists of strings of type $a^n b^m c^{m-n}$ with $m > n > 0$. By posing $k = m - n > 0$, hence $m = n + k$, those can be rewritten as $a^n b^{n+k} c^k$. Consider this grammar G_1 (axiom S):

$$G_1: \quad S \rightarrow A C \quad A \rightarrow a A b \mid a b \quad C \rightarrow b C c \mid b c$$

It holds $L(A) = a^n b^n$ and $L(C) = b^k c^k$ with $n, k > 0$, hence $L(S) = L(A) L(C) = a^n b^{n+k} c^k$, thus G_1 generates L_1 . On the other hand, it is well-known that the numerical relation between the exponents in $a^n b^{n+k} c^k$ is incompatible with REG. Thus the smallest family of L_1 is CF.

- Language L_2 is generated by grammar G_2 , already given. Apply to G_2 the expansion principle: replace rules $U \rightarrow b S$ and $T \rightarrow \varepsilon$ into rule $S \rightarrow a S U \mid T$, and so obtain rule $S \rightarrow a S b S \mid \varepsilon$, which generates the Dyck language over the parentheses $a b$. The remaining rules $T \rightarrow b T V \mid \varepsilon$ and $V \rightarrow c T$ of G_2 necessarily generate (sub)strings with one or more letters c . It soon follows that $L_2 \cap \{a, b\}^* = \text{Dyck}(a, b)$, which is not a regular language. Consequently language L_2 is not regular either, because if it were so, intersecting L_2 with the regular language $\{a, b\}^*$ would produce a regular language, not Dyck. Thus the smallest family of L_2 is CF.
- Language L_3 consists of strings of type $a^n b^m c^l$ with $m > 0$, $n > 1$ and $l > 0$. It is generated, for example, by the regular expression R below, thus its smallest family is REG:

$$R = a a^+ b^+ c^+$$

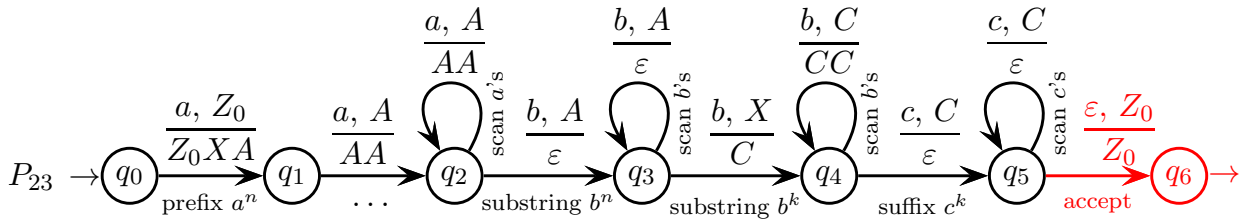
(b) Concerning how to design a recognizer automaton of the intersection language $L_2 \cap L_3$, note that:

- from grammar G_2 , in every string $w \in L_2$ the number of letters b is the sum of the numbers of letters a and c , i.e., $|w|_b = |w|_a + |w|_c$, regardless of the ordering of letters
- from the definition of L_3 (or from the regular expression R above), in every string $w \in L_3$ the left-to-right ordering of letters is $a \prec b \prec c$ with $|w|_b > 0$, $|w|_a > 1$ and $|w|_c > 0$
- hence every string $w \in L_2 \cap L_3$ is of the form $a^n b^{n+k} c^k$ with $|w|_a = n > 1$ and $|w|_c = k > 0$

A pushdown automaton (PDA) is a suitable recognizer for the intersection language $L_2 \cap L_3$. A PDA P_{23} such that $L(P_{23}) = L_2 \cap L_3$ should simply perform the following steps in sequence:

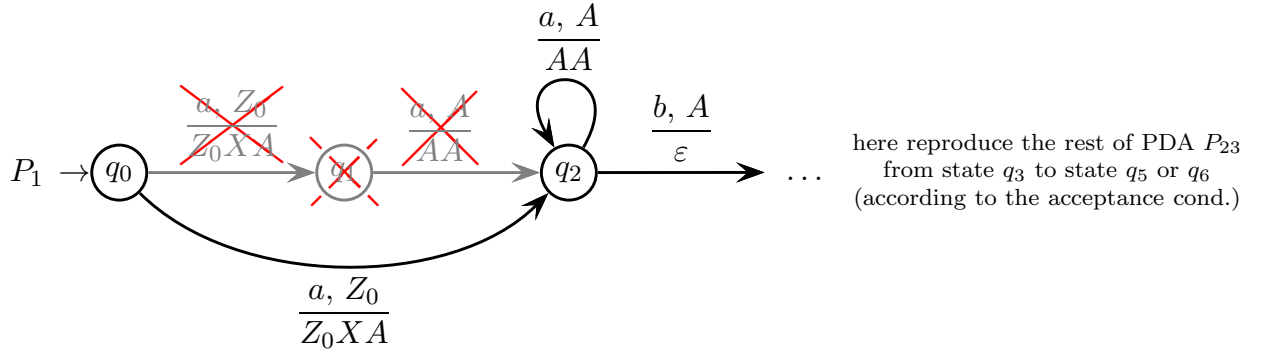
- 1) scan a prefix of letters a (at least two) and push one symbol A for each of them
- 2) scan a first substring of letters b and pop one symbol A for each of them
- 3) scan a second substring of letters b and push one symbol C for each of them
- 4) scan a suffix of letters c (at least one) and pop one symbol C for each of them
- 5) if the stack empties, accept the input scanned so far (go to a final state and halt)

The stack alphabet is $\Gamma = \{A, C, X, Z_0\}$ with bottom Z_0 . A set $Q = \{q_i \mid 0 \leq i \leq 6\}$ of seven (six) states is enough to serialize these steps, so as to grant the ordering of letters $a \prec b \prec c$ and the minimum numbers of two a 's and one c . Here is the state-transition graph of the PDA P_{23} :



States q_0 and q_6 are initial and final, respectively. Machine P_{23} is deterministic and accepts by final state. The pop move of symbol X separates the recognition of the substrings b^n and b^k , which handle the stack differently: b^n pops A^n and b^k pushes C^k . By removing state q_6 and its ingoing transition, machine P_{23} accepts by empty stack when the bottom Z_0 emerges in the state q_5 .

- (c) From the discussion at point (b), language L_1 is almost entirely contained in the intersection language $L_2 \cap L_3$, except for all its strings with only one letter a . These are exactly the strings $ab^k c^k$ with $k > 0$, which belong to L_1 but do not belong to the intersection $L_2 \cap L_3$. For example, let $w = abbc$, then $w \in L_1$, yet $w \notin L_2 \cap L_3$ since $w \notin L_3$, though $w \in L_2$. Therefore, the PDA P_{23} of point (b) recognizes language L_1 as well, with just one small change: redirect transition $q_0 \xrightarrow{\frac{a, Z_0}{Z_0 X A}} q_1$ to state q_2 , and remove both state q_1 and transition $q_1 \xrightarrow{\frac{a, A}{AA}} q_2$. Here is a draft of the PDA P_1 , derived from PDA P_{23} , that accepts language L_1 :



Now a single letter a is enough for the PDA P_1 to reach state q_2 and proceed with recognition from q_2 onwards, while PDA P_{23} needs two letters a . The rest of P_1 is unchanged with respect to P_{23} . For example, the string $abbc \in L_1$ shown above is accepted by P_1 , while it is rejected by P_{23} . In conclusion we have $L_1 = L(P_1)$ and machine P_1 is still deterministic. As with machine P_{23} , the acceptance condition can be easily changed by choosing between final state and empty stack.